Literature Survey: The Evolution of Privacy-Preserving Sealed-Bid Auctions

Aarav Oswal (230012) Ch. Rishitha (230311)
R. Charan (230819) Poorvie Sadagopan (230759)

CS670 Project Group
Department of Computer Science and Engineering
Indian Institute of Technology Kanpur

August 2026

Abstract

This literature survey traces the cryptographic evolution of privacy-preserving sealed-bid auctions along a single thread: progressively removing the trust that bidders must place in the auctioneer. We begin with Yao's Millionaires' Problem (1982), which reframes an auction as the secure computation of a maximum, and follow the arc to the two-server garbled-circuit construction of Naor, Pinkas, and Sumner (1999), which splits trust between two non-colluding servers. We then examine the practical deployment of secret-sharing multiparty computation in the work of Bogetoft et al. (2006, 2009), culminating in the Danish sugar-beet auction of January 2008, and turn under the hood to describe the bit-slice circuit of Kurosawa and Ogata (2002), the engine that extracts the winning bid -and, with a small modification, the second-highest bid from secret-shared inputs. Finally, we survey the modern verifiability frontier, in which commitments and zero-knowledge proofs (Galal–Youssef, 2018; zk-STARK-based schemes, 2024) render the auction outcome publicly verifiable even against a malicious auctioneer. Building on this trajectory, we propose to implement a semi-honest, sealed-bid auction supporting both first-price and second-price (Vickrey) mechanisms on the MP-SPDZ framework, with public verifiability against a malicious auctioneer deferred to future work.

1 Introduction: The Auctioneer's Dilemma

A sealed-bid auction promises secrecy: each bidder submits a private valuation, an auctioneer opens the bids, and a winner is declared. Unlike an open outcry auction, no bidder is supposed to learn what any other was willing to pay. Yet the moment the mechanism is instantiated, that promise frays — because the auctioneer, who exists precisely to open the bids, ends up learning exactly the information the mechanism was supposed to hide.

1.1 The Trust Problem

To decide who won and at what price, some party must look inside the sealed envelopes; once they have, the losing bids and everything they reveal about valuations, budgets, and strategy are no longer secret. A curious auctioneer can build a dossier of every bidder's willingness to pay. A corrupt one can leak the ordering to a favoured participant, or misreport the outcome in ways no losing bidder can detect. Even an honest auctioneer is a single point of failure whose compromise exposes every participant at once. The question that has driven three decades of cryptographic research is a natural one: can the same outcome be computed without requiring anyone including the auctioneer to see the individual bids at all?

The target can be stated in a single sentence: the protocol should reveal the winner and the price the winner pays, and nothing else. Two subtleties matter. First, the guarantee is stated *relative to the prescribed output*; some information is unavoidably implied by the outcome itself (a clearing price of p reveals that every other bid was at most p), and only leakage *beyond* what the output implies counts as a violation. Second, the guarantee must cover *partial* information: a protocol that hides bid values but reveals the full ranking of losers has still leaked substantial strategic information and fails the definition.

1.2 First-Price and Second-Price Auctions

Two variants dominate both the economic and the cryptographic literature. In a *first-price* sealed-bid auction, the highest bidder wins and pays their own bid. In a *second-price*, or *Vickrey*, auction, the highest bidder wins but pays the *second*-highest bid. The rules differ only in what the winner is charged, but induce very different strategic behaviour: in the Vickrey auction, truthful bidding submitting one's actual valuation-is a dominant strategy.

This last property is precisely where the traditional Vickrey auction is most vulnerable. Truthfulness is rational only if the auctioneer can be relied upon to report the second-highest bid honestly, and an auctioneer who knows every bid has both the means and the motive to inflate the clearing price toward the winner's true valuation. The Vickrey auction is thus a particularly clean illustration of the broader point: the trust assumption that makes classical sealed-bid auctions work is precisely the one cryptography aims to eliminate.

1.3 Roadmap

The remainder of this survey traces the evolution of privacy-preserving auctions along a single thread: at each stage, the role of the auctioneer is diminished. Section 2 introduces the cryptographic background and the reframing of an auction as the secure computation of a maximum. Section 3 presents the foundational two-server construction of Naor, Pinkas, and Sumner; Section 4 follows the arc to the deployed secret-sharing multiparty computation of Bogetoft et al. Section 5 then turns under the hood to describe the circuit that actually extracts the winner and, with a small modification, the second-highest bid from secret-shared inputs, and Section 6 surveys the modern frontier of publicly verifiable auctions secure against a fully malicious auctioneer. Section 7 consolidates the threat model, and Section 8 states the direction we propose to pursue.

2 Background: Secure Comparison and the Millionaires' Problem

Before turning to auction protocols proper, we introduce the cryptographic idea on which all of them rest: the possibility of computing a function on private inputs while revealing nothing beyond its output. This idea is due to Yao, and the observation that a sealed-bid auction is a natural instance of it is the conceptual hinge of the rest of the survey.

2.1 Yao's Millionaires' Problem

In 1982, Andrew Yao posed what has become the canonical thought experiment of secure computation [9]. Two millionaires wish to determine which of them is richer without revealing their actual wealth to each other or to any third party learning only the one bit they care about (who is richer) and nothing else (by how much).

Yao showed that this is possible, and in doing so introduced a general paradigm: any function expressible as a Boolean circuit can be evaluated on private inputs so that each party learns only the output. His construction used what would later be called a *garbled circuit* — one

party encodes the comparison as an encrypted circuit that the other evaluates, learning only the outcome. The Millionaires' Problem is the simplest non-trivial instance; the general problem it inaugurated, now known as *secure multiparty computation* (MPC), is the umbrella under which every construction in later sections sits.

2.2 An Auction is a Secure Maximum

The leap from Yao's problem to auctions is small but decisive. A first-price sealed-bid auction over n bidders is, stripped of its economic framing, the computation of a single function:

$$(i^\star, b^\star) \;=\; \arg\max_{i \in [n]} b_i, \qquad b^\star \;=\; \max_{i \in [n]} b_i,$$

where b_i is the bid submitted by bidder i. The Vickrey variant computes the same winning index $i^\star$ but returns the *second*-largest value. In either case the auctioneer's role is simply to evaluate this function on hidden inputs, and the privacy target of Section 1 - reveal the output and nothing else is precisely the guarantee MPC was invented to provide. An auction protocol is thus an instance of MPC in which the parties are the *bidders* and one or more *computing servers*, and the function happens to be a maximum.

2.3 The MPC Toolkit

Three primitives recur throughout the auction literature and are worth naming once, so later sections can invoke them without redefinition.

Secret sharing splits a value into shares distributed among several parties, such that any sufficiently large subset can reconstruct it while any smaller subset learns nothing. A bid can thus be held and computed on collectively, without any single party ever seeing it.

Garbled circuits, also due to Yao, let one party construct an encrypted circuit that a second party evaluates on encrypted inputs, learning only the output.

Oblivious transfer is a two-party primitive in which the sender holds several values and the receiver selects one, in such a way that the sender learns nothing about the choice and the receiver learns nothing about the unchosen values. It is the standard mechanism by which a party feeds private input into a garbled circuit.

3 The Foundational Construction: Garbled-Circuit Auctions

The first cryptographic construction to turn Yao's paradigm into a concrete auction protocol was proposed by Naor, Pinkas, and Sumner in 1999 [8]. Their scheme often abbreviated NPS gave the field its foundational architecture: instead of relying on a single trusted auctioneer, the role is split between two servers, and the auction is computed in such a way that neither server ever sees the individual bids. Almost every later construction in this report can be read as a response to some limitation of NPS, so it is worth setting out what the paper actually contributes.

3.1 Contribution in One Paragraph

It is easy to misread NPS as a paper about *how* to compute the winner on hidden bids. In fact its central contribution lies one layer above: an *architecture* in which a general-purpose secure computation any Boolean circuit, including whichever circuit extracts the winner can be evaluated on the bidders' private inputs without either computing party learning them. The winner-finding circuit itself is treated largely as a component; the paper's contribution is the cryptographic *container* in which such a circuit can safely be run.

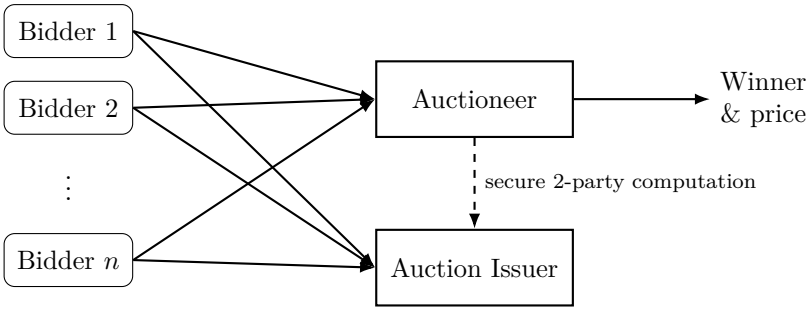


Figure 1: The NPS two-server architecture. Each bidder submits an encoded form of their bid to both computing parties; the auctioneer and the auction issuer then jointly evaluate the auction circuit, learning only the winner and the clearing price.

3.2 The Two-Server Architecture

NPS introduces two computing parties: an *auctioneer*, who manages the auction and eventually announces the outcome, and an independent *auction issuer*, who assists in the cryptographic setup. Bidders submit their bids to this pair jointly, in a form that neither party alone can decode. The auction is then executed as a secure two-party computation between the auctioneer and the issuer, producing the outcome without either seeing the bids.

The security guarantee is conditional: as long as the auctioneer and the auction issuer do not collude, no party including the computing parties themselves learns anything about the losing bids beyond what the outcome implies. Collusion between the two computing parties would collapse the guarantee entirely, but the hope is that the two roles can be assigned to organisations with divergent interests, so that collusion is contractually and practically implausible. Figure 1 summarises the setup.

3.3 The Cryptographic Mechanism

The two-server computation is instantiated with the two primitives introduced in Section 2: garbled circuits and oblivious transfer. The auction issuer prepares a garbled version of the auction circuit an encrypted description of the function that takes the bids as input and outputs the winner and clearing price and hands it to the auctioneer, who evaluates it. The bidders' inputs must be fed into this encrypted circuit without revealing them to the auctioneer; NPS achieves this using a variant of oblivious transfer that they introduce for this purpose, called *proxy oblivious transfer*, in which the bidder acts as sender, the issuer as chooser, and the auctioneer as receiver of the encrypted input labels.

The overall effect is that the auctioneer evaluates the entire auction circuit on encrypted inputs, learning only the output; the issuer, having constructed the garbled circuit but never seen the bidders' input labels, learns nothing at all. Only the two computing parties acting together could reconstruct the bids, and by assumption they do not.

3.4 What NPS Guarantees, and What It Does Not

Under its trust assumption, NPS achieves the privacy target of Section 1: the outcome of the auction is revealed and nothing more. The construction is generic in the auction rule first-price, second-price, and other sealed-bid variants can all be encoded as the underlying circuit and its round complexity is low, making it attractive on paper.

Its guarantees rest, however, on two assumptions that later work has found increasingly uncomfortable. The first is the *non-collusion* of the two computing parties: privacy collapses

entirely if the auctioneer and the issuer share their views. The second, less obvious, is that both parties are *honest-but-curious* - they follow the protocol exactly and merely try to learn from what they see. If either party actively deviates from the protocol, NPS provides no mechanism to detect it. A subsequent analysis by Juels and Szydlo [6] identified a concrete instantiation of this weakness, showing that a cheating server can undetectably alter the outcome of the auction; a follow-up construction closed the specific gap but preserved the underlying two-server model.

Both limitations are architectural rather than incidental. They motivate the shift, taken up in the next section, from a two-party container of assumed non-colluding servers to a genuinely distributed computation among several parties, in which trust is spread more broadly and no small subset can compromise the outcome on its own.

4 From Theory to Deployment: Secret-Sharing Multiparty Computation

The construction of Naor, Pinkas, and Sumner (Section 3) demonstrates that a sealed-bid auction can be evaluated without either computing party observing the bids, but its guarantee rests on a two-party architecture whose security collapses entirely under collusion. This section follows the literature's response to that limitation: a shift from garbled circuits to *secret-sharing* multiparty computation, distributed across several computing servers rather than two, and its realisation in a genuine large-scale deployment [2, 3].

4.1 From Two Servers to a Distributed Computation

Where NPS splits trust between exactly two parties, the constructions of Bogetoft et al. distribute it across n computing servers, of which some threshold t may be corrupt without compromising privacy; typical instantiations tolerate one corruption among three servers, or two among five. Rather than encoding the auction as a single garbled circuit evaluated by one party, each bid is *secret-shared* (Section 2) across all n servers, and the servers jointly compute on these shares - addition and multiplication by a public constant require no communication at all, general multiplication requires one round, and comparison, the operation an auction depends on most, requires a dedicated protocol operating bit-by-bit on the shared values. This generalisation from two parties to n removes the single point of failure inherent in non-collusion between exactly two servers, at the cost of a more involved cryptographic toolkit.

4.2 The Auction Protocol

The 2006 paper targets a more general auction than the single-item maximum of Section 2: a *double auction*, in which many buyers and sellers each submit a bid specifying the quantity they are willing to trade at each of a discrete set of prices. Because aggregate supply increases and aggregate demand decreases monotonically in price, the *market clearing price* — the price at which supply and demand balance - can be located by binary search over the price grid, requiring only $O(\log L)$ secure comparisons for a grid of L prices. Only the outcome of each comparison is revealed during the search; the individual bids remain shared throughout, and the clearing price itself is opened only at the end. Figure 2 summarises this lifecycle.

The 2006 paper reports a prototype implementation benchmarking this approach for 32-bit integers across several (n, t) server configurations, concluding that a price grid of roughly two thousand points - requiring on the order of eleven comparisons was comfortably feasible in practice. The 2009 paper's principal technical contribution is a substantially faster comparison protocol: for numbers of the size used in the deployment below, it requires fewer than twenty communication rounds, against more than one hundred for the previously best constant-round construction.

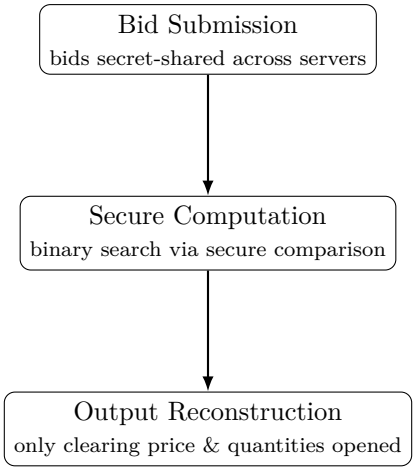


Figure 2: The protocol lifecycle of Bogetoft et al.: bids are shared across servers, the servers jointly locate the market clearing price via secure comparison, and only the final outcome is reconstructed.

4.3 Deployment: The Danish Sugar Beet Auction

The 2009 paper reports the deployment of this protocol at national scale. Danish sugar beet farmers held production contracts with Danisco, the country's sole processor, and reforms to European Union sugar subsidies created an urgent need to reallocate these contracts toward the most efficient producers. A nationwide double auction was the natural mechanism, but no single acceptable auctioneer existed: Danisco risked misusing bid information in ongoing contract negotiations, while excluding Danisco entirely was impractical, since it administers the contracts being traded.

The resolution was to distribute the auctioneer's role across three mutually distrusting parties - Danisco, the farmers' growers' association (DKS), and the research team implementing the system using the secret-sharing protocol above, so that no single party ever observed an individual bid. The computation was run on 14 January 2008, processing bids from 1,229 farmers over a grid of 4,000 possible prices, resolved using approximately twelve secure comparisons; as a result, roughly 25,000 tons of production rights changed ownership. The dominant cost in practice was not the comparison protocol itself but the decryption of secret-shared bids, since over a thousand bidders generated on the order of nine million individual values to process. The authors report this as the first large-scale, practical deployment of secure multiparty computation, and note that a post-auction survey found the large majority of farmers considered bid confidentiality important and were satisfied with the guarantees the system provided.

4.4 Security Assumptions and Limitations

The deployed protocol assumes a *semi-honest*, honest-majority adversary: computing servers are trusted to follow the protocol correctly, and are relied upon only not to collude in interpreting what they legitimately observe. This assumption was considered reasonable for the sugar beet deployment, where the three computing parties had no serious expectation that the others would actively deviate, but none was willing to accept sole custody of the sensitive data.

This assumption is also the construction's principal limitation. If a server deviates actively from the protocol rather than merely observing passively, or if collusion exceeds the assumed threshold, the privacy guarantee no longer holds and, more importantly for what follows, there is no mechanism by which an outside party could detect that this had happened. Extending the guarantees of this section to a fully malicious auctioneer, whose misbehaviour can be publicly caught rather than merely assumed away, is the subject of Section 6.

5 The Core Engine: Securely Computing the Maximum (and Second-Maximum)

Section 3 described NPS's contribution as a container: a two-server architecture in which some circuit computing the auction outcome can be evaluated without either party seeing the bids. What that circuit actually does (how it extracts a maximum, and with a small modification a second-maximum, from n secret values) was left unspecified. This section fills in that gap. The construction we describe, due to Kurosawa and Ogata [7], is not tied to any single trust model. As the literature on auctioneer-free protocols illustrates, the same underlying scan can plug into radically different architectures, from server-based secret sharing to bidder-only bulletin-board protocols; Section 5.6 compares two such instantiations directly.

5.1 From Pairwise Comparisons to a Single Scan

The naive way to find the maximum of n secret bids is to compare them pairwise: run a secure comparison (an instance of Yao's Millionaires' Problem from Section 2) between every pair, and track which bidder wins each comparison. Even in the more efficient tournament-elimination variant, this approach is dominated by the overhead and round complexity of repeated secure comparisons.

The key realisation behind the bit-slice construction is that a maximum does not require pairwise comparison at all. It can instead be computed by a single coordinated scan across all n bids simultaneously, examining one bit position at a time. Kurosawa and Ogata report that their bit-slice circuit is roughly seven times faster than the previously best construction in the same setting.

5.2 The Bit-Slice Algorithm

Fix a bit length k, and suppose each bidder i holds a secret k-bit bid b_i, with bits $b_i[k-1]$ (most significant) down to $b_i[0]$ (least significant). The algorithm maintains a *surviving-bidder mask* $S = (S_1, \ldots, S_n)$, a vector of n bits, one per bidder, initialised to all 1: every bidder begins as a candidate for the maximum.

The scan proceeds from the most significant bit down to the least significant. At bit position j:

1. **Check.** Compute whether any surviving bidder has a 1 at position j, that is, compute the OR, over all i, of $(S_i \wedge b_i[j])$.
2. **Update.** If the check is 1 (some survivor has a 1 at this position), eliminate every survivor whose bit at position j is 0: for each i, set $S_i \leftarrow S_i \wedge b_i[j]$. If the check is 0, leave S unchanged and proceed to the next bit.

After all k positions have been scanned, exactly the bidder or bidders holding the maximum bid remain marked in S. Intuitively, the mask enforces lexicographic elimination: at each bit position, any surviving bidder who could not possibly hold the maximum (because a rival survivor has a 1 where they have a 0) is dropped, and the comparison never needs to be repeated for eliminated bidders at later positions.

Crucially, both operations in the inner loop (the OR check and the mask update) depend only on bit values and the current mask, never on any bid in the clear. They can therefore be implemented as secure boolean operations on shared or encrypted data. This is what makes the scan architecture-agnostic: the algorithm does not care how the underlying secure AND and OR are realised, only that they *can* be realised in the chosen trust model.

5.3 Worked Example

Table 1 traces the algorithm on three bidders with 4-bit bids: $A = 1011$ (11), $B = 1101$ (13), $C = 1001$ (9).

Table 1: Bit-slice scan over bidders A, B, C. Once a bidder is eliminated (here, A and C after bit position 2), their bits at later positions no longer affect the mask; this is indicated with a dash. At bit position 1, B's bit is 0, so the check fails and the mask is simply carried forward unchanged.

Bit (j)	$A[j]$	$B[j]$	$C[j]$	Any survivor = 1?	Mask after update
3 (MSB)	1	1	1	yes	$\{A, B, C\}$
2	0	1	0	yes	$\{B\}$
1	—	0	—	no	$\{\}$
0	—	1	—	-	$\{\}$

The scan correctly identifies B as the winner after only four bit-level rounds, without ever comparing A to B or A to C directly.

5.4 Complexity

At each of the k bit positions, the algorithm performs n secure AND operations (the mask update) and a single n-input secure OR (the survivorship check). Both reduce, at the level of secure computation, to a constant number of secure multiplications or additions per bidder per bit. The total cost is therefore $O(nk)$ secure operations, linear in both the number of bidders and the bid length. This asymptotic figure is matched by a well-optimised tournament of pairwise comparisons, but the bit-slice construction wins on constants and round complexity: the entire computation is a single top-down scan rather than a cascade of $\log n$ comparison layers, and the number of secure gates is a small multiple of nk rather than a much larger one.

5.5 From First-Price to Vickrey

The scan as described yields the maximum: the winner and, if desired, their bid value, giving the first-price outcome. The second-price (Vickrey) outcome requires the second-highest bid.

The natural extension is an oblivious re-scan that removes the maximum's contribution from consideration without revealing the winner mid-protocol; the details vary across the literature, but the common idea is that the winning bidder's influence on the second scan is cancelled inside the secure computation itself, so that only the final winner identity and clearing price are opened. The asymptotic cost roughly doubles to $O(2nk)$, which remains linear and does not change the overall picture. In our worked example, an oblivious second scan over the same three bidders would return A's bid (11) as the second-highest value; in a Vickrey auction, B would win and pay 11.

5.6 The Same Engine, Different Architectures

The scan of Section 5.2 is agnostic to how its two per-bit operations (a secure AND and a secure n-input OR) are actually computed. Different trust models instantiate them differently, and the literature offers examples at both extremes of the trust spectrum. It is worth being precise about what is, and is not, shared between these examples: the deployed protocol of Bogetoft et al. (Section 4) locates a market clearing price via binary search using secure two-value comparisons, which is a related but distinct algorithm from the n-bidder simultaneous scan below. The comparison here is at the level of the underlying boolean primitives (secure AND and OR on shared bits), not a claim that the two protocols run identical circuits.

Secret-sharing instantiation (Bogetoft-style). In a secret-sharing deployment of the scan itself, of the kind used by Bogetoft et al. for their comparison protocol, each bid bit is secret-shared across the computing servers, and the AND and OR operations are realised by secure multiplication and addition on shares (for instance, using the identity $\mathrm{OR}(x, y) = 1-(1-x)(1-y)$ to build n-input ORs from repeated ANDs). Nothing is reconstructed until the final winner and clearing price are opened at the end. This style of secret-sharing MPC is the same family of techniques that powered the Danish sugar-beet auction, though that deployment's own comparison protocol proceeds by binary search rather than by running the bit-slice scan described above.

Auctioneer-free instantiation (SEAL). A very different architectural choice appears in the SEAL protocol of Bag et al. [1], which dispenses with computing servers altogether and has the bidders themselves execute the scan. The n-input OR at each bit position is computed via a modified Anonymous Veto network (AV-net) [5]. Each bidder publishes two Diffie-Hellman public keys in a first round; in the second round, each bidder broadcasts a cryptogram built from one of two independent generators depending on their bit: a "0-cryptogram" raised against the first generator, or a "1-cryptogram" raised against the second. The product of all second-round broadcasts reveals whether *any* bidder voted 1, without revealing *who*. No secret channels and no computing servers are required, and every step is publicly verifiable on a bulletin board. In this setting, the AND step of the scan requires no interaction at all (each bidder can compute $S_i \wedge b_i[j]$ locally from public information and their own private bits), so only the OR check requires the AV-net exchange. Figure 3 illustrates the two-round exchange for a single bit position.

The point of holding these two constructions side by side is not to prefer one over the other but to make visible what the bit-slice engine is doing at a higher level of abstraction: it defines a way to find the maximum of hidden bids in terms of two boolean primitives, and the choice of trust model determines how those primitives are instantiated. A secret-sharing MPC and an auctioneer-free bidder-only protocol look nothing alike as systems, but at the algorithmic level they can be seen as running the same kind of scan.

Table 2 summarises the contrast.

Table 2: Two literature instantiations of the same bit-slice scan. The algorithm of Section 5.2 is identical in both; only the per-bit AND and OR mechanisms, and with them the entire trust architecture, differ.

	Secret sharing	SEAL (AV-net)
Trust model	servers, honest majority	no servers, bidders only
Per-bit AND	secure multiplication over shares	local, no interaction
Per-bit OR	secure multiplication over shares	AV-net broadcast
Communication	low; shares sent once	broadcast per bit
Demonstrated scale	$\approx$ 1200 bidders (Bogetoft deployment)	moderate n
Public verifiability	no	yes

5.7 Handoff

Both instantiations above assume, as NPS did, that the parties executing the protocol are honest-but-curious: they follow it as specified and merely try to learn from what they observe. Neither guarantees anything if a party deviates from the protocol outright, nor gives any outside observer a way to check that the announced winner is correct without trusting the parties who computed it. Section 6 takes up this remaining gap: how commitments and zero-knowledge proofs can render the outcome publicly verifiable even against a fully malicious party.

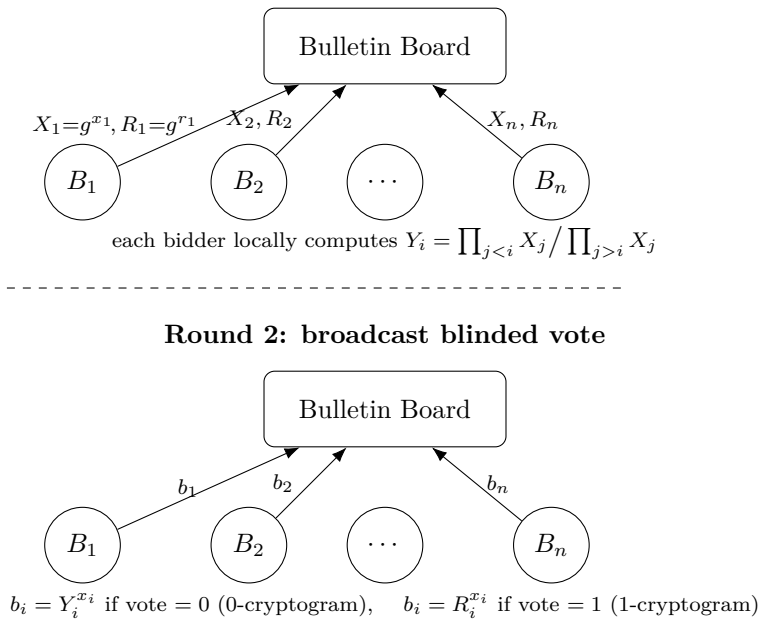


Figure 3: The two-round modified AV-net protocol used inside SEAL to compute the survivorship OR at a single bit position. Round 1 establishes fresh Diffie-Hellman keys $X_i = g^{x_i}$ and $R_i = g^{r_i}$ for each bidder. Round 2 broadcasts a blinded vote built from one of two independent generators: Y_i (derived from the X_j values of the other bidders) for a 0-vote, or the bidder's own R_i for a 1-vote. The product $\prod_i b_i$ equals the group identity 1 if and only if every bidder voted 0, revealing the OR outcome without revealing any individual vote. This differs from the original Hao-Zielinski AV-net, which encodes a 1-vote by multiplying in a fixed generator rather than switching to a second one; SEAL's two-generator variant is what allows the disjunctive zero-knowledge proofs of well-formedness to be constructed. This two-round exchange is repeated once per bit position, with each bidder's vote for position j computed locally as $S_i \wedge b_i[j]$.

6 The Frontier: Verifying a Malicious Auctioneer

The constructions of Sections 3 and 4 remove the auctioneer's ability to *see* the bids, but they still assume it *computes honestly*. A reader arriving here naturally asks what happens if that assumption fails - and the literature on verifiable auctions is the answer.

6.1 From Curious to Dishonest

A semi-honest auctioneer follows the protocol and merely tries to learn from what it sees. A *malicious* auctioneer may deviate from the protocol altogether, and it can produce a wrong outcome even without ever seeing a bid: by feeding incorrect values into the secure computation, corrupting or dropping protocol messages, or simply announcing a winner it never actually computed. Hiding the inputs, in other words, secures privacy but says nothing about correctness. This motivates a different goal - letting anyone *verify* that the announced outcome is the one the protocol should have produced.

6.2 Commitments and Zero-Knowledge Proofs

Two primitives make such verification possible. A *commitment* is a cryptographic sealed envelope: publishing it binds its author to a fixed value (which cannot later be changed) while hiding that value until it is opened. Bidders can thus publicly commit to their bids without revealing them, yet be unable to alter them afterwards.

A *zero-knowledge proof* lets one party prove that a statement is true without revealing anything beyond its truth. An auctioneer can, in principle, prove that its announced winner is consistent with the committed bids - that the protocol was executed correctly - while revealing nothing about the bids themselves.

6.3 Galal–Youssef: Verifiable Auctions via Commitments and ZK

The scheme of Galal and Youssef [4] was among the first to assemble these primitives into a verifiable sealed-bid auction. Bidders publish commitments to their bids on a smart contract and then reveal the openings to the auctioneer, who determines the winner and produces zero-knowledge proofs that the claimed outcome is correct. Because the commitments and proofs live on-chain, the result is *publicly verifiable*: anyone not only the bidders can check that the auctioneer did not cheat, even a fully malicious one.

This buys correctness against a dishonest auctioneer, but at a cost. The auctioneer receives the openings of the losing bids in the clear, and so learns exactly the information that the constructions of the previous sections were built to hide. Verifiability, in this design, is obtained by giving up privacy against the auctioneer.

6.4 Verifiability Without Sacrificing Privacy

This leaves a natural question. The multiparty constructions of Section 4 keep the bids hidden from the computing parties, while the verifiable schemes above prove the outcome correct but expose the losing bids to the auctioneer. Can both guarantees hold at once - an outcome that is publicly verifiable, computed by parties that never see the bids? Combining input privacy of the multiparty kind with a zero-knowledge proof of correct execution is an active direction in the literature, and it is the point at which the trust placed in the auctioneer - both to keep bids secret and to report the result honestly - would finally be removed in full.

7 Threat Model and Trust Assumptions

The preceding sections traced a range of threat models across the literature. This section consolidates the specific model this project targets - the adversary, the trust architecture, and the precise privacy guarantee.

7.1 Adversary and Trust Architecture

The project targets the *semi-honest* (honest-but-curious) adversary: every party follows the protocol as specified but may try to learn additional information from its view of the execution. The stronger *malicious* model, in which parties may deviate arbitrarily from the protocol, is deferred to Section 8.

The computation is distributed across a small set of non-colluding computing servers holding secret shares of the bids (Section 4). Privacy holds as long as no more than a specified threshold of these servers collude; collusion above the threshold falls outside the model.

7.2 Per-Party View

Bidders. Follow the protocol and submit valid bids, but may try to infer other bids from their own input and the public output.

Computing servers. Execute the MPC honestly on secret-shared bids and may try to learn bid values from their local view. Under the non-collusion assumption, no subset below the threshold can reconstruct any bid.

External observers. See only the public output; individual network channels carry secret shares, which convey nothing on their own.

7.3 Privacy Target

The protocol reveals the identity of the winner and the clearing price, and nothing else about any bid - including no partial information such as the rank, ordering, or approximate value of any losing bid - under the adversary and non-collusion assumptions above.

Two subtleties are already implicit in this statement but worth naming. First, information logically implied by the output itself (that every losing bid is at most the clearing price in the first-price case, or that the clearing price equals the second-highest bid in the Vickrey case) is inherent to the mechanism and is not counted as a leak. Second, the guarantee is stated *relative to the output*: it is not an absolute claim of bid secrecy but the claim that nothing beyond the output is revealed.

8 Proposed Direction

Building on the trajectory traced above, we propose to implement a sealed-bid auction in the secret-sharing multiparty-computation style of Bogetoft et al. (Section 4), supporting both the first-price and second-price mechanisms as a single circuit whose output is either the maximum or the second-maximum of the secret-shared bids (Section 5). The construction targets the semi-honest, non-colluding adversary model consolidated in Section 7, in which the protocol reveals only the winner and the clearing price. Several natural extensions lie outside the scope of the initial implementation and are left as possible future directions: lifting the security to a fully malicious auctioneer, adding public verifiability of the outcome through the commitment- and zero-knowledge-based techniques surveyed in Section 6, and exploring an auctioneer-free instantiation of the bit-slice engine along the lines of the SEAL protocol discussed in Section 5, which removes the computing servers entirely at the cost of a different communication profile.

Together, these define the endpoint of the arc this survey has traced the point at which no trust in any single party is required at all.

Presentation

A recorded presentation accompanying this report is available at:

https://youtu.be/Sp1i0Rik87c

References

[1] Samiran Bag, Feng Hao, Siamak F. Shahandashti, and Indranil G. Ray. SEAL: Sealed-bid auction without auctioneers. *IEEE Transactions on Information Forensics and Security*, 15: 2042–2052, 2020. doi: 10.1109/TIFS.2019.2955793.

[2] Peter Bogetoft, Ivan Damgård, Thomas Jakobsen, Kurt Nielsen, Jakob Pagter, and Tomas Toft. A practical implementation of secure auctions based on multiparty integer computation. In *Financial Cryptography and Data Security (FC)*, volume 4107 of *Lecture Notes in Computer Science*, pages 142–147. Springer, 2006. doi: 10.1007/11889663_10.

[3] Peter Bogetoft, Dan Lund Christensen, Ivan Damgård, Martin Geisler, Thomas Jakobsen, Mikkel Krøigaard, Janus Dam Nielsen, Jesper Buus Nielsen, Kurt Nielsen, Jakob Pagter, Michael Schwartzbach, and Tomas Toft. Secure multiparty computation goes live. In *Financial Cryptography and Data Security (FC)*, volume 5628 of *Lecture Notes in Computer Science*, pages 325–343. Springer, 2009. doi: 10.1007/978-3-642-03549-4_20.

[4] Hisham S. Galal and Amr M. Youssef. Verifiable sealed-bid auction on the ethereum blockchain. In *Financial Cryptography and Data Security Workshops (FC)*, volume 10958 of *Lecture Notes in Computer Science*, pages 265–278. Springer, 2019. doi: 10.1007/978-3-662-58820-8_18.

[5] Feng Hao and Piotr Zieliński. A 2-round anonymous veto protocol. In *Security Protocols Workshop*, volume 5087 of *Lecture Notes in Computer Science*, pages 202–211. Springer, 2009. doi: 10.1007/978-3-642-04904-0_28.

[6] Ari Juels and Michael Szydlo. A two-server, sealed-bid auction protocol. In *Financial Cryptography (FC)*, volume 2357 of *Lecture Notes in Computer Science*, pages 72–86. Springer, 2003. doi: 10.1007/3-540-36504-4_6.

[7] Kaoru Kurosawa and Wakaha Ogata. Bit-slice auction circuit. In *Computer Security — ESORICS 2002*, volume 2502 of *Lecture Notes in Computer Science*, pages 24–38. Springer, 2002. doi: 10.1007/3-540-45853-0_2.

[8] Moni Naor, Benny Pinkas, and Reuban Sumner. Privacy preserving auctions and mechanism design. In *Proceedings of the 1st ACM Conference on Electronic Commerce (EC)*, pages 129–139. ACM, 1999. doi: 10.1145/336992.337028.

[9] Andrew C. Yao. Protocols for secure computations. In *Proceedings of the 23rd Annual Symposium on Foundations of Computer Science (FOCS)*, pages 160–164. IEEE Computer Society, 1982. doi: 10.1109/SFCS.1982.38.